

Aprendizado de Maquina

Bernardo Klein - 22103012
Bernardo Fiorese - 23102185
Bruno Almeida - 22180462
Bruno Roesse - 24280180
João Aiolfi - 22107503
Rafael Prestes - 22280060

30 de junho de 2026

Resumo

Este trabalho compara três abordagens para o *matching* automático entre consultas textuais heterogêneas (ex.: FANTA LARANJA 2L C/6) e um catálogo normalizado de 14.206 produtos: (1) NLP clássico, com TF-IDF (palavras e n-gramas de caracteres) e BM25 implementado do zero; e (2) Deep Learning, com embeddings semânticos do modelo multilíngue *paraphrase-multilingual-MiniLM-L12-v2* (Sentence-Transformers), nas variantes busca densa e híbrida (BM25 top-50 $\rightarrow$ re-ranqueamento neural); e (3) re-ranqueamento por LLM (Google Gemini) nos modos zero-shot e few-shot, escolhendo o produto entre os candidatos do BM25. No conjunto de teste (250 queries), a melhor variante clássica (TF-IDF de caracteres) atingiu $P@1 = 0,992$, $MRR@5 = 0,996$ e $R@5 = 1,000$, superando a melhor variante neural ($P@1 = 0,944$). A análise qualitativa mostra que o pré-processamento é decisivo para ambas as abordagens (sem ele, a neural cai de $P@1$ 0,920 para 0,520 na validação) e que os erros remanescentes se concentram em variantes quase idênticas do mesmo produto (multipack, sabor, cor, tamanho). Concluímos que, em um catálogo de vocabulário bem controlado, os métodos clássicos bem pré-processados oferecem a melhor relação entre simplicidade, custo e precisão.

Descrição do Problema

O problema consiste em identificar automaticamente, a partir de um texto de consulta escrito de forma heterogênea e sem padronização, qual é o produto correspondente em um catálogo normalizado. Trata-se de um problema clássico de recuperação de informação, com alta relevância prática em sistemas de compras, gestão de estoque e integração entre bases de dados de fornecedores distintos.

1. Referencial Teórico

1.1 TF-IDF e Similaridade de Cosseno

O TF-IDF (Term Frequency–Inverse Document Frequency) representa documentos como vetores esparsos em que o peso de cada termo cresce com sua frequência no documento (TF) e decresce com sua frequência na coleção (IDF), penalizando termos comuns e valorizando termos discriminativos. A similaridade entre query e documento é medida pelo cosseno do ângulo entre seus vetores, que é independente do comprimento dos textos. Utilizamos duas granularidades: palavras (uni e bigramas) e

n-gramas de caracteres (3 a 5, delimitados por palavra), sendo esta última conhecida por sua robustez a erros ortográficos e variações morfológicas — características típicas das queries deste problema.

O peso de um termo t em um documento d é dado por

$$w(t, d) = \text{tf}(t, d) \cdot \text{idf}(t), \quad \text{idf}(t) = \ln \frac{1 + N}{1 + \text{df}(t)} + 1, \quad (1)$$

onde $\text{tf}(t, d)$ é a frequência do termo no documento (na variante de caracteres usamos a forma sublinear $\text{tf} = 1 + \ln f(t, d)$), N é o número de documentos e $\text{df}(t)$ o número de documentos que contêm t (a suavização “+1” segue a implementação do scikit-learn). Os vetores resultantes são normalizados em norma L_2 . A similaridade entre os vetores da query $\mathbf{q}$ e de um produto $\mathbf{d}$ é o cosseno do ângulo entre eles:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \sum_t \hat{w}(t, q) \hat{w}(t, d), \quad (2)$$

sendo a última igualdade válida porque os vetores $\hat{w}$ estão normalizados.

1.2 BM25

O BM25 (Okapi) é uma função de ranqueamento probabilística que estende a intuição do TF-IDF com dois refinamentos: saturação da frequência do termo (parâmetro k_1) e normalização pelo comprimento do documento (parâmetro b). É o método clássico padrão em motores de busca e costuma superar o TF-IDF em textos curtos. Implementamos o BM25 do zero (src/bm25.py) com índice invertido, $k_1 = 1,5$ e $b = 0,75$:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (3)$$

1.3 Embeddings Semânticos e Sentence-Transformers

Modelos de linguagem baseados na arquitetura Transformer, como o BERT, produzem representações contextuais densas do texto. A biblioteca Sentence-Transformers ajusta esses modelos com redes siamesas para que a similaridade de cosseno entre embeddings de sentenças reflita similaridade semântica. Diferentemente do TF-IDF/BM25 (vetores esparsos, casamento lexical), embeddings capturam sinonímia e proximidade semântica mesmo sem sobreposição de palavras. Usamos o modelo multilíngue paraphrase-multilingual-MiniLM-L12-v2 (384 dimensões, suporte a português), que roda localmente em CPU, sem custo de API.

1.4 LLMs: matching zero-shot e few-shot

Uma terceira família de métodos trata o matching como uma tarefa de seleção em linguagem natural: fornece-se a um *large language model* (LLM) a consulta e uma lista de candidatos pré-filtrados (por BM25) e pede-se que ele escolha o produto correspondente; assim, o LLM atua como re-ranqueador. No modo *zero-shot*, o prompt contém apenas a instrução e os candidatos; no modo *few-shot*, incluem-se alguns exemplos resolvidos no próprio prompt (*in-context learning*), o que orienta o formato da resposta e ajuda nos casos difíceis. Diferentemente das abordagens anteriores, um LLM pode raciocinar explicitamente sobre os atributos e responder “nenhum”, oferecendo rejeição nativa de NO_MATCH; em contrapartida, agrega custo e latência por consulta e exige cuidado com a reprodutibilidade.

2. Delimitação do Desenvolvimento

Implementado pelo grupo:

- Todo o pré-processamento textual, implementado em `src/preprocess.py`, incluindo normalização de unidades, tratamento de embalagens, dicionário de abreviações e remoção de stopwords;
- O algoritmo BM25 completo, implementado em `src/bm25.py`, com índice invertido e sem uso de bibliotecas prontas de BM25;
- Os dois pipelines de matching e suas variantes, implementados em `src/approach1_classic.py` e `src/approach2_deep.py`, incluindo a estratégia híbrida BM25 → re-rank neural e o cache de embeddings;
- O script de avaliação, implementado em `src/evaluate.py`, com as métricas P@1, MRR@5 e R@5;
- A exploração dos dados, as análises qualitativas e o preenchimento do arquivo `queries.csv`, por meio dos scripts `src/explore.py`, `src/qualitative.py`, `src/run_no_match.py` e `src/fill_queries.py`.
- A variante de re-rank por LLM com prompting zero-shot e few-shot sobre os candidatos do BM25, implementada em `src/approach2_llm.py`: construção dos prompts, parsing da resposta, tratamento do limite de requisições (retry com backoff) e lógica de rejeição de NO_MATCH.

Reaproveitado de bibliotecas públicas:

- `TfidfVectorizer` e `cosine_similarity`, da biblioteca *scikit-learn*;
- O modelo pré-treinado `paraphrase-multilingual-MiniLM-L12-v2`, utilizado por meio da biblioteca *Sentence-Transformers*;
- As bibliotecas *pandas* e *numpy*, utilizadas para manipulação e processamento dos dados.
- A API do Google Gemini (modelo `gemini-2.5-flash-lite`), acessada via biblioteca *google-generativeai* e usada apenas como decisor na variante LLM — nenhum dado de treino foi enviado nem houve fine-tuning.

Nenhum código de matching foi copiado de repositórios de terceiros.

3. Metodologia

A seção a seguir aborda a metodologia seguida pelo grupo para o desenvolvimento do trabalho. Ela descreve toda a parte de exploração dos dados, estratégias usadas no pré-processamento, os pipelines seguidos para cada abordagem e as métricas de avaliação utilizadas.

3.1 Exploração dos dados

O catálogo tem 14.206 produtos (1.833 marcas), com 29 nomes duplicados sob ids diferentes (fonte de ambiguidade irreduzível). O arquivo `queries.csv` tem 16.441 consultas não rotuladas (mediana de 7 tokens), com forte presença de tokens de embalagem (C/12, C/6, UND, 12X350ML, CX, FD — 1064 ocorrências) e abreviações. Os conjuntos de validação e teste (250 queries cada) têm 100% dos `matched_id` presentes no catálogo (não há NO_MATCH rotulado) e estilo mais descritivo que o de `queries.csv`. Identificamos ainda famílias inteiras de queries NO_MATCH em `queries.csv`: marcas como Qualitá (1.234 ocorrências), Taeq, Swift, Carnilove e Mormaii não existem no catálogo.

3.2 Pré-processamento

Aplicado tanto às queries quanto aos nomes do catálogo (nome + marca, quando a marca ainda não aparece no nome):

Transformação	Exemplo
Minúsculas + remoção de acentos	Guaraná → guarana
Expansão de <i>s/</i> e <i>c/</i>	S/ACUCAR → sem acucar
Remoção de quantidades de embalagem	C/6, CX12, 6 UND → (removido)
Multipack mantém o tamanho unitário	12X350ML → 350ml
Canonização de unidades	2L, 2 litros → 2l; 15 GR → 15g
Abreviações não ambíguas	CERV → cerveja; REFRIG → refrigerante
Pontuação e stopwords (de, do, com, por...)	removidas

Duas decisões merecem destaque: (i) mantivemos a palavra *sem* (não a tratamos como stopword), pois ela é discriminativa em “sem açúcar”, “sem gás”, “sem lactose”; (ii) o dicionário de abreviações é deliberadamente conservador, abreviações ambíguas como COND (condensado × condicionador) e SAB (sabão × sabonete) ficaram de fora, pois a expansão errada induz matches errados.

3.3 Pipelines

Abordagem 1 (clássica). Para cada query pré-processada, calcula-se a similaridade com todos os 14.206 produtos e retornam-se os top-5. Três variantes: TF-IDF de palavras (1–2 gramas), TF-IDF de caracteres (3–5, *char_wb*, com *sublinear_tf*) e BM25 próprio.

Abordagem 2 (deep learning). Os 14.206 produtos são codificados uma única vez pelo modelo neural (105,6 s em CPU; embeddings cacheados em disco). Duas variantes: *densa* (cosseno entre o embedding da query e todos os produtos); *híbrida* (recomendada no enunciado) BM25 seleciona 50 candidatos e o modelo neural decide o ranking final, reduzindo o espaço de busca e o custo.

Abordagem 3 (LLM, zero/few-shot). O BM25 seleciona os 10 candidatos mais prováveis e um LLM atua como decisor final. O modelo Google Gemini (*gemini-2.5-flash-lite*) recebe a consulta e a lista numerada de candidatos, retornando apenas o identificador do produto correspondente ou 0, quando julga que nenhum candidato é adequado (rejeição de NO_MATCH). Avaliaram-se duas variantes de prompting: *zero-shot* (sem exemplos no prompt) e *few-shot* (com quatro exemplos consulta→produto correto, extraídos da validação e excluídos da amostra avaliada para evitar vazamento de dados). Como o modelo retorna apenas um vencedor, o ranking final é composto pela escolha do LLM na primeira posição e pelos demais candidatos na ordem original do BM25, preservando as métricas MRR@5 e R@5. Devido ao limite de requisições da camada gratuita da API, essa variante foi avaliada em uma amostra de 30 consultas da base de validação, em vez das 250 consultas completas.

Desempate determinístico. Produtos com score idêntico (ex.: nomes duplicados no catálogo, ou versões unitária × multipack que se tornam iguais após o pré-processamento) são ordenados de forma estável pela ordem do catálogo. Sem isso, o resultado dependeria de um desempate arbitrário do algoritmo de ordenação, detectamos variações de até $\pm 0,8$ ponto percentual em $P@1$ entre implementações de ordenação diferentes, o que comprometeria a reprodutibilidade.

Protocolo de avaliação. Todas as decisões (escolha de variantes, ablações, hiperparâmetros) foram tomadas exclusivamente sobre *queries_val.csv*. O conjunto *queries_test.csv* foi usado uma única vez, ao final, para as métricas definitivas.

3.4 Métricas de avaliação

Para avaliar a qualidade dos métodos propostos, utilizaram-se as três métricas definidas no enunciado: $P@1$, $MRR@5$ e $R@5$. Em todas elas, considera-se que, para cada consulta, existe apenas um produto correto no catálogo.

Seja N o número de consultas avaliadas e seja r_i a posição (indexada a partir de 1) em que o produto correto aparece para a consulta i . Caso ele não esteja entre os cinco primeiros resultados, considera-se $r_i = \infty$. Utiliza-se ainda $1[\cdot]$ como função indicadora, que assume valor 1 quando a condição é verdadeira e 0 caso contrário.

$$P@1 = \frac{1}{N} \sum_{i=1}^N 1[r_i = 1] \quad (4)$$

$$MRR@5 = \frac{1}{N} \sum_{i=1}^N \frac{1}{r_i} 1[r_i \leq 5] \quad (5)$$

$$R@5 = \frac{1}{N} \sum_{i=1}^N 1[r_i \leq 5] \quad (6)$$

A interpretação de cada métrica é a seguinte:

- **$P@1$ (Precisão no Top-1):** mede a porcentagem de consultas em que o produto correto aparece na primeira posição. Por exemplo, um valor de $P@1 = 0,98$ significa que, em média, 98% das consultas tiveram o produto correto como primeira resposta.
- **$MRR@5$ (Mean Reciprocal Rank):** avalia quão próximo do topo o produto correto aparece entre os cinco primeiros resultados. Quanto mais alta sua posição, maior a pontuação. Por exemplo, se o produto estiver em 1º lugar, a contribuição é 1; em 2º lugar, $1/2 = 0,5$; em 3º lugar, $1/3 \approx 0,33$; e assim sucessivamente. Caso não apareça entre os cinco primeiros, a contribuição é zero.
- **$R@5$ (Recall no Top-5):** mede a porcentagem de consultas em que o produto correto aparece em qualquer posição entre os cinco primeiros resultados. Por exemplo, um valor de $R@5 = 1,00$ indica que o sistema sempre conseguiu recuperar o produto correto no Top-5, mesmo que nem sempre na primeira posição.

Neste trabalho, $P@1$ foi considerada a métrica principal, pois o objetivo é retornar um único produto ao usuário. O $MRR@5$ permite avaliar a qualidade do ranqueamento quando o primeiro resultado não é o correto, enquanto o $R@5$ indica a capacidade do sistema de recuperar o produto correto entre os principais candidatos. Todas as métricas são calculadas pelo script `src/evaluate.py`.

4. Resultados e Análise

Nesta seção mostramos os resultados das três abordagens — as duas clássicas/neurais e a variante baseada em LLM. Começamos pelos números de validação (Seção 4.1), que usamos durante o desenvolvimento para comparar as variantes e escolher a melhor de cada abordagem; é também nessa seção que apresentamos a Abordagem 3 (re-ranqueamento por LLM), avaliada em uma amostra reduzida por restrições da API. Depois vêm os números de teste (Seção 4.2), que medimos só uma vez, no final, para registrar o resultado oficial. Em seguida, olhamos exemplos de acertos e erros (Seção 4.3) — incluindo a capacidade de rejeição (`NO_MATCH`) que só o LLM oferece — e comparamos custo e desempenho das três abordagens (Seção 4.4).

4.1 Validação (desenvolvimento) — 250 queries

A Tabela 1 mostra como cada uma das cinco variantes se saiu nas 250 consultas de validação. Foi com base nesses números que escolhemos a melhor versão de cada abordagem (TF-IDF de caracteres na Abordagem 1 e a versão híbrida na Abordagem 2). Já dá para notar o padrão que se repete no teste: as abordagens clássicas ficam na frente em $P@1$, e em todas elas o $R@5$ é quase 1,000 — ou seja, o produto certo quase sempre aparece entre os cinco primeiros; o que falta é colocá-lo em primeiro lugar.

Tabela 1: Validação (desenvolvimento): cinco variantes nas 250 consultas.

Variante	$P@1$	$MRR@5$	$R@5$	Tempo (250 q)
TF-IDF palavras (1–2)	0,952	0,972	1,000	1,1 s
TF-IDF caracteres (3–5)	0,972	0,983	1,000	12,5 s
BM25 ($k_1 = 1,5$; $b = 0,75$)	0,964	0,980	1,000	0,3 s
Neural densa	0,920	0,945	0,984	7,3 s
Neural híbrida (BM25 top-50 $\rightarrow$ re-rank)	0,924	0,952	0,992	6,6 s

Abordagem 3 (LLM) — amostra de 30 consultas. A variante baseada em LLM exige uma ressalva de escopo: por causa do limite de requisições da camada gratuita da API do Google Gemini, ela foi avaliada apenas em uma amostra de 30 consultas da validação, e não nas 250 da Tabela 1. Nessa abordagem, o BM25 recupera os 10 candidatos mais prováveis e o modelo `gemini-2.5-flash-lite` seleciona o que considera correspondente (ou 0, indicando `NO_MATCH`); o ranking final coloca a escolha do LLM em primeiro lugar, seguida pelos demais candidatos na ordem original do BM25. Avaliamos dois estilos de *prompting*, *zero-shot* e *few-shot*. Para uma comparação justa, a Tabela 2 traz também o BM25 medido exatamente na mesma amostra de 30 consultas.

Tabela 2: Validação: variante baseada em LLM na amostra de 30 consultas ($n = 30$).

Abordagem	$P@1$	$MRR@5$	$R@5$
BM25 (referência)	0,967	0,983	1,000
LLM zero-shot	0,967	0,983	1,000
LLM few-shot	0,967	0,983	1,000

As três linhas são idênticas. Em todas as 30 consultas o LLM concordou com o primeiro colocado do BM25, sem alterar nenhuma decisão de ranking; por isso o único erro da amostra — a fralda, em que o candidato correto não se distingue da versão com outra contagem de unidades — permaneceu inalterado. O *few-shot* não superou o *zero-shot*, sugerindo que as consultas avaliadas não eram ambíguas o bastante para que exemplos adicionais mudassem a decisão do modelo. E, como o LLM só enxerga os candidatos já recuperados pelo BM25, sua capacidade de correção fica limitada pelo *recall* dessa etapa: quando o BM25 já posiciona o produto certo em primeiro lugar, não há o que re-ranquear. A única funcionalidade realmente nova trazida pelo LLM — a rejeição de consultas sem correspondência — é discutida na análise qualitativa (Seção 4.3).

4.2 Teste (métricas definitivas) — 250 queries

Depois de escolher as variantes na validação, medimos o resultado final nas 250 consultas de teste. A Tabela 3 traz esses números, junto com o tempo, o custo e a complexidade de cada abordagem. A Abordagem 3 não aparece nessa tabela: pelo mesmo limite de API, o re-ranqueamento por LLM não

foi medido no conjunto de teste. Como na amostra compartilhada de validação ele apenas reproduziu o BM25 (Seção 4.1), seu teto neste conjunto é o próprio ranking do BM25, e por isso a comparação definitiva se concentra nas Abordagens 1 e 2.

Tabela 3: Teste (métricas definitivas): abordagens nas 250 consultas, com tempo, custo e complexidade.

Abordagem	$P@1$	$MRR@5$	$R@5$	Tempo	Custo	Complex.
Ab. 1 — TF-IDF caracteres	0,992	0,996	1,000	12,2 s	Gratuito (CPU)	Baixa
Ab. 1 — BM25 (própria)	0,988	0,992	1,000	0,2 s	Gratuito (CPU)	Baixa
Ab. 1 — TF-IDF palavras	0,932	0,958	0,996	1,1 s	Gratuito (CPU)	Baixa
Ab. 2 — Neural densa	0,944	0,962	0,984	8,7 s	Gratuito (local)	Média
Ab. 2 — Neural híbrida	0,944	0,962	0,984	6,6 s*	Gratuito (local)	Média

As variantes densa e híbrida são estatisticamente equivalentes no teste (na execução de referência empatam em $P@1 = 0,944$; diferem por no máximo uma query, $\leq 0,4$ p.p.), a cascata BM25→neural não traz ganho de acurácia neste conjunto, pois o pré-filtro top-50 raramente descarta um acerto que a busca densa traria.

Nota de reprodutibilidade. Re-executando os scripts em ambiente com versões mais recentes das bibliotecas (pandas 3.0, numpy 2.5, scikit-learn 1.9, torch 2.12, sentence-transformers 5.6), toda a Abordagem 1 e a busca densa reproduzem *exatamente* os números das Tabelas 1 e 3; a variante híbrida reproduz dentro de ≤ 1 query ($\leq 0,4$ p.p.), variação atribuível ao desempate em queries de fronteira sob versões diferentes das bibliotecas. Nenhuma conclusão do trabalho se altera.

4.3 Análise qualitativa

Acertos. Ambas as abordagens resolvem com folga queries descritivas com a mesma base lexical do catálogo, mesmo com ordem de palavras diferente (“Mortadela Fatiada Defumada Sadia Soltíssimo 200g” → “Mortadela defumada fatiada soltíssimo sadia 200g”) e queries abreviadas após normalização (“CORONA CERO SUNBREW LONG NECK 330ML - C/24” é resolvida pelo TF-IDF-char).

Erros da Abordagem 1 (apenas 2 no teste). “Fralda Pampers Confort Sec G 128 Unidades” → previu a versão de 60 unidades: nosso pré-processamento remove “<n> unidades” por ser, em geral, multiplicidade de embalagem — mas em fraldas a contagem é o tamanho do produto, e as duas versões ficam idênticas após a normalização (empate exato; o correto ficou em 2º). “Espumante Casa Valduga Arte Tradicional Brut 750ml” → previu *brut rosé* em vez de *brut branco*: a query não especifica a cor — caso genuinamente ambíguo.

Erros da Abordagem 2 (14 no teste), em três padrões:

- *Variante/sabor errado* (9 casos): “Refresco em Pó Maracujá Tang” → “...manga Tang”; “Colgate Total 12 Antitártaro” → “... gengiva reforçada”. Embeddings aproximam sabores/variantes semanticamente próximos, perdendo o detalhe lexical que decide o match;
- *Tamanho/quantidade errada* (4 casos): “Salsicha Viena Ceratti 500g” → “Salsicha Viena Perdigão 500g” (priorizou o peso, errou a marca);
- *Categoria errada por proximidade semântica* (1 caso): “Pipoca para Microondas YOKI” → “Saco hermético para freezer e microondas” — o termo dominante no embedding (“microondas”) arrasta o match para outra categoria.

Comparação direta (teste). Em 12 queries o clássico acertou onde o neural errou; em nenhuma o neural acertou onde o clássico errou; 2 erros em comum (a fralda e o espumante, ambos ambíguos). Não identificamos um tipo de query em que a abordagem neural seja sistematicamente melhor neste dataset — o vocabulário do catálogo é controlado e, após a normalização, o casamento lexical é quase sempre suficiente. A vantagem teórica dos embeddings (sinonímia: “refrigerante de cola” $\approx$ “coca-cola”) raramente é exigida pelas queries avaliadas. O re-ranqueamento por LLM (Abordagem 3) não muda esse quadro: restrito aos candidatos do BM25 e concordando com ele em todas as 30 consultas de validação, não acrescentou poder de correção neste catálogo.

Casos ambíguos. O catálogo tem 29 nomes duplicados sob ids distintos (ex.: “Adoçante líquido sucralose linea 75ml” com dois EANs) — nesses casos nenhum sistema distingue os ids apenas pelo texto. No teste, os empates relevantes são versões unitárias $\times$ multipack (“...lata 350ml” $\times$ “...lata 350ml 12 un”, Δ score = 0,0000), contagens removidas pelo pré-processamento (fralda 128 $\times$ 60 unidades) e variantes de cor não especificadas na query (brut branco $\times$ rosé). Com o desempate determinístico pela ordem do catálogo, a versão unitária (que aparece primeiro) é preferida — comportamento razoável para um sistema de pedidos.

Casos NO_MATCH. Val/teste não contém NO_MATCH, mas queries.csv contém famílias inteiras (Qualitá, Taeq, Swift, Carnilove, Mormaii, Skol comum...). Rodamos as duas abordagens nessas queries (results/no_match.md): nenhuma tem mecanismo nativo de rejeição — ambas sempre devolvem algum produto. A diferença está nos scores. No TF-IDF-char, queries NO_MATCH de estilo descritivo ficam entre 0,24 e 0,68, bem abaixo do p5 dos matches corretos na validação (0,880). Já na neural os scores continuam altos em NO_MATCH (até 0,90 para o limpador Qualitá, que tem substitutos semanticamente próximos), tornando o limiar pouco confiável. O limiar depende do estilo da query — em queries abreviadas de queries.csv, um match correto pode pontuar $\approx$ 0,73 (“FANTA LARANJA 2L C/6”) enquanto NO_MATCHes ficam em $\approx$ 0,41–0,43; para esse estilo, um limiar de $\approx$ 0,55 separa razoavelmente os regimes. Em produção, o limiar deve ser calibrado com uma amostra rotulada de cada estilo, enviando para revisão humana o que ficar abaixo.

Rejeição nativa (Abordagem 3). Entre as três abordagens, só o LLM tem um mecanismo nativo de rejeição: pode responder 0 (NO_MATCH) quando julga que nenhum candidato corresponde. Em testes manuais ele rejeitou corretamente consultas de marcas ausentes do catálogo, como Mormaii e Taeq. Esse ganho, porém, vem com um comportamento conservador: o modelo também rejeitou FANTA LARANJA 2L C/6, que tinha correspondente válido — um falso negativo, justamente na consulta de baixo score discutida acima. Ou seja, a rejeição existe, mas a versão gemini-2.5-flash-lite ainda erra para o lado de recusar matches legítimos; um uso confiável pediria um modelo mais robusto ou ajustes de *prompting*. As abordagens clássicas e neural, por não rejeitarem nada, dependem de um limiar de score calibrado (como descrito acima) para emular essa função.

Preenchimento de queries.csv. Conforme a descrição dos dados (“matched_id — a ser preenchido pelo grupo”), aplicamos a variante vencedora (TF-IDF-char) em lote às 16.441 queries: results/queries_preenchido.csv traz text, matched_id (top-1) e score, em 9,4 s ($\approx$ 1.700 queries/s). O arquivo original em Dados/ não foi modificado. A coluna de score permite filtrar possíveis NO_MATCH (23% das queries pontuam abaixo de 0,45).

4.4 Tradeoff simplicidade $\times$ desempenho

Neste problema, o método mais simples é também o mais preciso: o TF-IDF-char vence em $P@1$ (0,992) e o BM25 entrega 0,988 com custo computacional $\approx$ 60 $\times$ menor (0,2 s vs. 12,2 s) e zero dependências pesadas. A abordagem neural agrega complexidade (download de modelo de 470 MB, PyTorch, indexação) sem ganho de acurácia aqui — mas seria a escolha certa se as queries usassem vocabulário livre/sinônimos distantes do catálogo, ou em cenários multilíngues. A variante híbrida confirma a recomendação do enunciado: re-ranquear só os top-50 do BM25 melhora a $R@5$ da neural na validação (0,992 vs. 0,984) e reduz o tempo por query.

A Abordagem 3 (LLM) ocupa o extremo de custo e complexidade do espectro: a camada gratuita da API levou vários minutos para avaliar apenas 30 consultas, contra os poucos segundos que o BM25 gasta para centenas, e ainda assim não melhorou as métricas — na amostra avaliada apenas reproduziu o BM25. O fator limitante não foi a compreensão de linguagem do modelo, mas a granularidade dos atributos disponíveis para distinguir produtos parecidos (peso, contagem de unidades, cor), que nenhum modelo recupera do texto quando ele simplesmente não está presente. O único motivo para pagar esse custo seria a rejeição nativa de `NO_MATCH` (Seção 4.3); mesmo assim, valeria a pena só com um modelo mais robusto e se o tratamento de consultas sem correspondência fosse prioridade do sistema.

5. Conclusão, Melhorias e Dificuldades

5.1 Conclusões

A partir dos experimentos realizados, destacam-se as seguintes conclusões:

1. **Competitividade de Métodos Clássicos:** Em catálogos com vocabulário controlado, os métodos clássicos bem pré-processados mostraram-se extremamente competitivos. O modelo TF-IDF baseado em caracteres atingiu uma precisão no topo-1 de $P@1 = 0,992$ no conjunto de teste, enquanto o algoritmo BM25 próprio alcançou $P@1 = 0,988$, apresentando um tempo de resposta altamente eficiente de aproximadamente 1 ms por *query*.
2. **Impacto do Pré-processamento:** O estágio de pré-processamento domina o resultado final em ambas as abordagens (clássica e neural). Ele é o responsável por traduzir o jargão dos pedidos textuais (como `C / 6`, `12X350ML`, `CERV`) para o vocabulário padronizado do catálogo. A ausência desse componente resulta em uma degradação severa no desempenho da abordagem neural, com uma perda de até 40 pontos em $P@1$.
3. **Natureza dos Erros Remanescentes:** Os erros restantes não decorrem de uma falha de “entendimento” semântico dos modelos, mas sim de problemas associados à granularidade da informação (como identificação de *multipack*, sabor, cor e contagem). Trata-se de atributos cuja resolução exige a aplicação de regras de negócio específicas — tais como a extração e comparação estruturada de quantidades e sabores em campos distintos —, e não necessariamente o emprego de modelos de linguagem maiores.
4. **Avaliação do Re-ranqueamento por LLM:** O experimento com LLM nas variantes *zero-shot* e *few-shot* confirmou, de forma negativa, a mesma hipótese observada nas abordagens anteriores: sobre os candidatos previamente recuperados pelo BM25, o modelo apenas reproduziu as escolhas já corretas, sem proporcionar ganho de acurácia. O único diferencial identificado foi a capacidade de rejeição de consultas `NO_MATCH`, embora sua confiabilidade tenha se mostrado insuficiente na versão gratuita do modelo utilizada.

5.2 Melhorias Futuras

Como possíveis evoluções do sistema desenvolvido, destacam-se as seguintes propostas:

- **Especialização do modelo de *embeddings*:** Atualmente foi utilizado um modelo pré-treinado de uso geral. Como melhoria, seria possível realizar seu *fine-tuning* utilizando os pares de consulta e produto corretos presentes no arquivo `queries.csv`, permitindo que os *embeddings* representem melhor as particularidades do domínio de supermercados.

- **Re-ranqueamento supervisionado:** Em vez de utilizar um LLM em *zero-shot* ou *few-shot*, que não apresentou ganhos neste trabalho, uma alternativa seria treinar um *Cross-Encoder* supervisionado. Diferentemente do LLM, esse modelo aprenderia diretamente, a partir de exemplos rotulados, como diferenciar produtos muito semelhantes, como variantes de sabor, cor, tamanho ou *multipack*.
- **Extração estruturada de atributos:** Muitos dos erros restantes decorrem de diferenças em atributos específicos, como quantidade, sabor ou número de unidades por embalagem. Assim, uma etapa dedicada à extração dessas informações poderia ser utilizada após o ranking para eliminar candidatos incompatíveis e melhorar a decisão final.
- **Aprimoramento da detecção de NO_MATCH:** O único benefício observado no uso do LLM foi sua capacidade de identificar consultas sem correspondente no catálogo. Trabalhos futuros podem explorar modelos de linguagem mais robustos e estratégias de *prompting* específicas para tornar essa decisão mais confiável e reduzir a ocorrência de falsos negativos.
- **Combinação entre BM25 e embeddings:** Neste trabalho, os métodos foram utilizados em cascata. Como alternativa, pode-se combinar diretamente os escores produzidos pelo BM25 e pela busca vetorial, permitindo que evidências léxicas e semânticas contribuam simultaneamente para o ranking final.

5.3 Dificuldades Encontradas

Durante o desenvolvimento, mapearam-se os seguintes desafios e limitações técnicas:

- **Ambiguidade em Abreviações:** Presença de termos ambíguos (como COND e SAB) cuja expansão textual automatizada não possui margem de segurança estatística.
- **Perda de Informação por Limpeza Textual:** A remoção ingênua de padrões como «n> unidades" sob a premissa de se tratar apenas de embalagem eliminou informações cruciais para a diferenciação em categorias de produtos específicas, tais como fraldas descartáveis.
- **Variantes e Duplicações:** Existência de variantes *multipack* com descrições de texto unitário idênticas, além de itens com nomes duplicados no próprio catálogo corporativo.
- **Instabilidade por Empates de Score:** Cenários de empate nos *scores* de similaridade demandaram a adoção de um algoritmo de desempate puramente determinístico para assegurar a reprodutibilidade dos testes, visto que variações na ordenação causavam flutuações de até $\pm 0,8$ p.p. na métrica $P@1$.
- **Incompatibilidade de Ambiente (Windows):** Ocorrência de conflitos de DLL entre as bibliotecas `pandas` e `PyTorch` no ambiente operacional Windows. O problema foi contornado ao fixar rigorosamente a ordem de importação dos pacotes no código-fonte, solução devidamente documentada no arquivo `README.md`.
- **Limitações da API do LLM:** O limite de requisições da camada gratuita da API do Google Gemini (aproximadamente 5–15 requisições por minuto, além de uma cota diária) tornou a avaliação da abordagem baseada em LLM significativamente mais lenta, restringindo os experimentos a uma amostra da validação e exigindo a implementação de mecanismos de tratamento de *rate limit*, como tentativas automáticas (*retry*) com *backoff* exponencial.